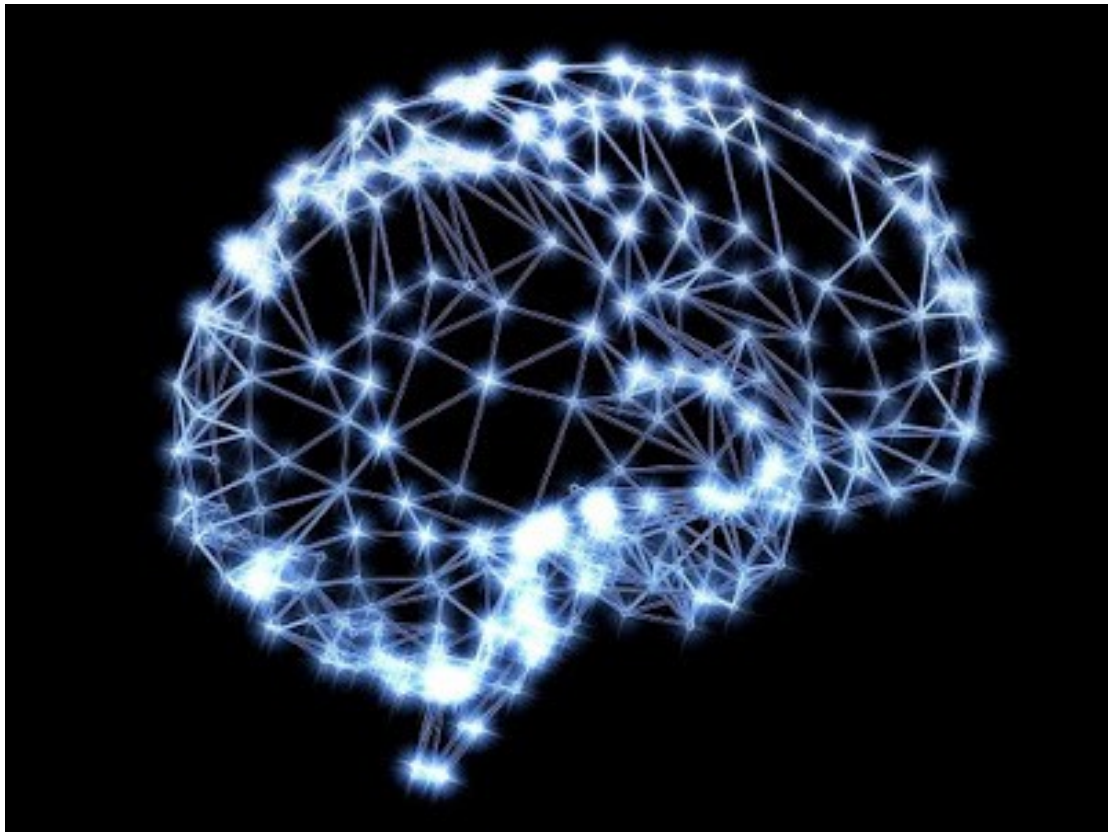
BSAN 750 Data Mining and Machine Learning

Lecture 7. Neural Networks

Shaobo Li

University of Kansas

Background



Human brain

- Receive inputs
- Processing electric and chemical signals
- Neuron switching time: ~ 0.001 second
- Number of neurons: $\sim 10^{11}$
- Connections per neuron: $\sim 10^4 - 10^5$
- Scene recognition time: ~ 0.1 second
- Much parallel computation

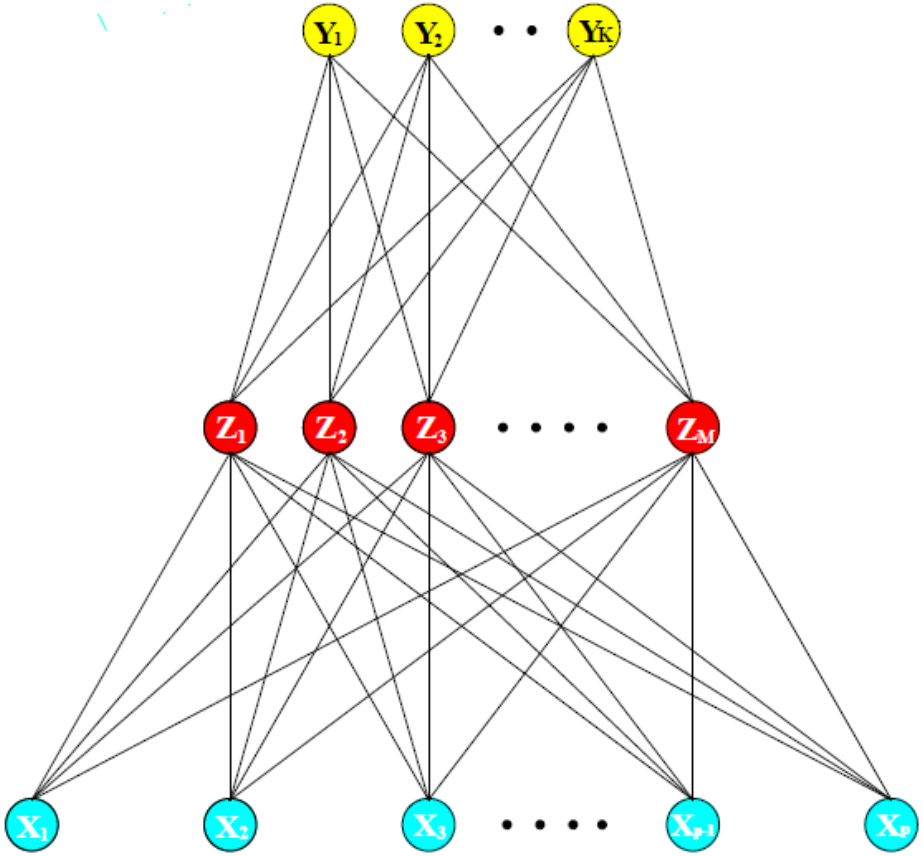
Question:

To recognize a image, how many neurons switch in your brain?

Artifical Neural Network (ANN)

- Input: $X_1, X_2, \ldots, X_p$
- Linear combination: $w_0 + w_1 X_1 + w_2 X_2 + \ldots + w_p X_p$
  The bias w_0 and weights w_j are the parameter in the system.
- Pass to first hidden layer: could be many neurons
  A (nonlinear) transformation function $\sigma(w_0 + \sum_{j=1}^{p} w_j X_j)$, also called activation function.
- Pass to the second hidden layer
- .....
- Output layer: one (for regression) or more (for classification)

An Illustraion (one hidden layer ANN)



Question: How does ANN learn?

How does it learn? – e.g. linear unit

- Suppose that $\sigma(\cdot) = \mathbb{I}(\cdot)$ is identity function
- Loss function: squared error

$$Err(w) = \frac{1}{2}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

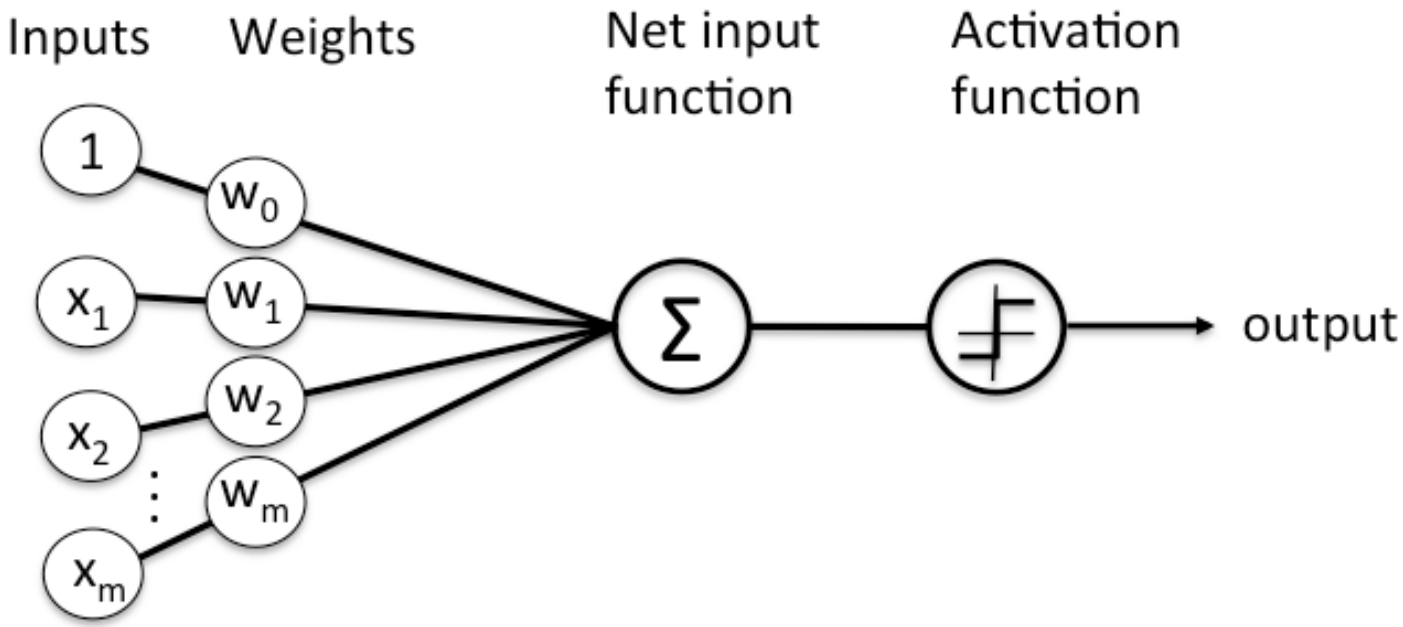
- Optimization algorithm: gradient descent

$$\nabla E(w) = \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \ldots, \frac{\partial E}{\partial w_p},\right]$$

- Training rule:

$$w^{new} = w^{old} - \eta \nabla E(w^{old})$$

A simple neural network – Perceptron



A simple neural network – Perceptron

- Developed in 1960s
- A single layer
- A step function

$$\sigma(w_0 + \sum_{j=1}^{p} w_j X_j) = \begin{cases} 0 & \text{if } w_0 + \sum_{j=1}^{p} w_j X_j \leq 0 \\ 1 & \text{if } w_0 + \sum_{j=1}^{p} w_j X_j > 0 \end{cases}$$

- A linear binary classifier
- Works well for linearly separable data

Question: Why is the threshold 0?

Network of perceptron

- Multiple neurons
- One or more hidden layers
  a single perceptron does not have hidden layers
- Can represent a wide range of nonlinear system
- But, the step function is discontinuous!
  So what?

Sigmoid Neural Networks

- Approximate the discontinuous step function with a continuous function
- Sigmoid function is a popular one

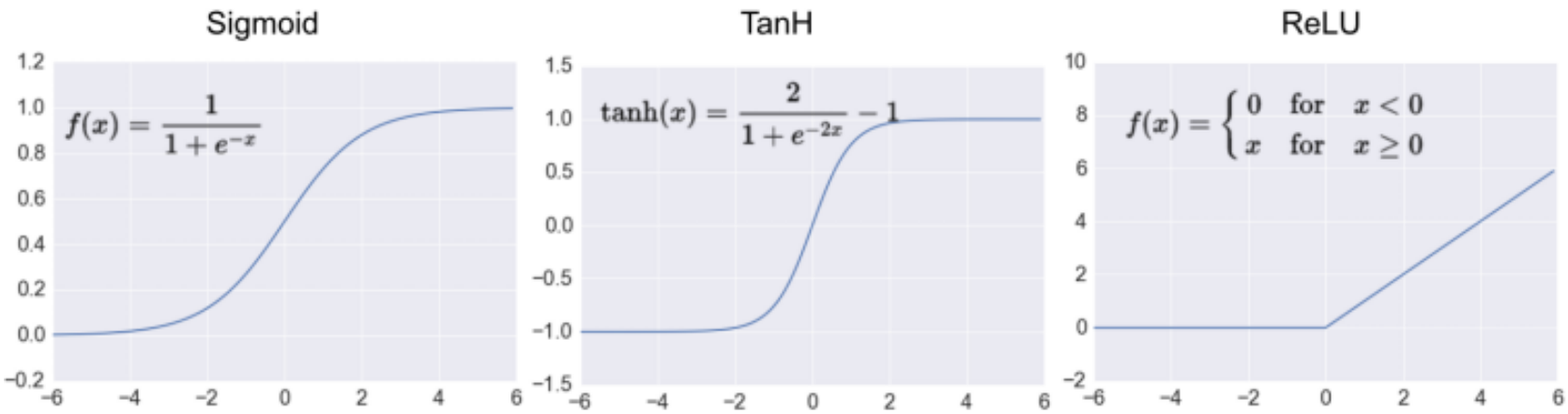
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Continuous and differentiable!

$$\sigma'(x) = \sigma(x) \times (1 - \sigma(x))$$

- A single layer is equivalent to logistic regression

Other popular activation funtions



Backpropagation

- Developed in 1970s
- The “black box" that people talked
- All about partial derivatives
- Chain rule
- Well explained in this video.

Chain rule

- $y = f(x)$; $z = g(y)$; $s = h(z)$
- In one equation, $s(x) = h(g(f(x)))$
- Derivative with respect to x: $\frac{\partial s}{\partial x} = \frac{\partial s}{\partial z}\frac{\partial z}{\partial y}\frac{\partial y}{\partial x}$
- An example:
  $y = x + 2$; $z = e^y$; $s = z^2 - z$
  Then, $s(x) = e^{2(x+2)} - e^{x+2}$
  $s'(x) = ?$
  Using chain rule, did you get the same result?

Backpropagation for neural networks

- E.g. one hidden layer and three neurons, a single output unit
- Let our loss function be squared loss

$$L(y, \hat{y}) = \frac{1}{2} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

- $\hat{y}_i$ is the output from output layer (2nd layer), and it is simply the function value of the activation function.

$$\hat{y}_i = \sigma(w_1^{(2)} a_{1i} + w_2^{(2)} a_{2i} + w_3^{(2)} a_{2i} + b^{(2)})$$

Backpropagation for neural networks (cont'd)

- a_{1i}, a_{2i}, a_{3i} are the activation function values in the hidden layer (1st layer)

$$a_{1i} = \sigma(w_{11}^{(1)} x_{1i} + w_{21}^{(1)} x_{2i} + w_{31}^{(1)} x_{3i} + b_1^{(1)})$$
$$a_{2i} = \sigma(w_{12}^{(1)} x_{1i} + w_{22}^{(1)} x_{2i} + w_{32}^{(1)} x_{3i} + b_2^{(1)})$$
$$a_{3i} = \sigma(w_{13}^{(1)} x_{1i} + w_{23}^{(1)} x_{2i} + w_{33}^{(1)} x_{3i} + b_3^{(1)})$$

- x_i are the feature input for ith observation.
- Using matrix expression

$$\mathbf{a}_i = \sigma(W^{(1)} \mathbf{x}_i + \mathbf{b}^{(1)})$$

Backpropagation for neural networks (cont'd)

- Look at the output layer first

$$\frac{\partial L_i}{\partial \hat{y}_i} = -(y_i - \hat{y}_i); \quad \frac{\partial \hat{y}_i}{\partial z_i^*} = \sigma'(z_i^*),$$

where $z_i^* = w_1^{(2)} a_{1i} + w_2^{(2)} a_{2i} + w_3^{(2)} a_{2i} + b^{(2)}$

- How the change of weights (2nd layer) affect loss

$$\begin{aligned}\frac{\partial L_i}{\partial w_1^{(2)}} &= \frac{\partial L_i}{\partial \hat{y}_i} \times \frac{\partial \hat{y}_i}{\partial z_i^*} \times \frac{\partial z_i^*}{\partial w_1^{(2)}} \\ &= \underbrace{-(y_i - \hat{y}_i) \times \sigma'(z_i^*)}_{\delta^{(2)}} \times a_{1i}\end{aligned}$$

similar for $w_2^{(2)}, w_3^{(2)}$ and $b^{(2)}$.

- Using gradient descent to update $w_1^{(2)}, w_2^{(2)}, w_3^{(2)}$ and $b^{(2)}$

Backpropagation for neural networks (cont'd)

- Now update the weights at the hidden layer (first layer) given the updated $\mathbf{w}^{(2)}$ and $b^{(2)}$

$$\frac{\partial L_i}{\partial w_{11}^{(1)}} = \frac{\partial L_i}{\partial \hat{y}_i} \times \frac{\partial \hat{y}_i}{\partial z_i^*} \times \frac{\partial z_i^*}{\partial a_{1i}} \times \frac{\partial a_{1i}}{\partial z_{1i}} \times \frac{\partial z_{1i}}{\partial w_{11}^{(1)}}$$
$$= \underbrace{\delta^{(2)} \times w_1^{(2)} \times \sigma'(z_{1i})}_{\delta_1^{(1)}} \times x_1$$

- similar for all rest $w^{(1)}$'s and $\mathbf{b}^{(1)}$
- Using gradient descent to update the parameters.
- Exercise:
  - Rewrite all these using matrix form for all observations.
  - Generate it to multiple hidden layers.
  - Implement the algorithm in R.

Deep Learning

- Representation learning
- Multilayer neural networks
- Abandoned by ML community in 1990s
- Regains interests in 2006 by researchers in Canadian Institute for Advanced Research (CIFAR)
- Some popular deep learning systems:
  - Convolutional neural networks
  - Recurrent neural networks
  - Generative neural networks
- A good article everyone should read.